CSI101 COMPUTER SYSTEMS

Lab 4 — A Function That Uses the Stack

Assembly Programming · Practical application 11.4 — A function using the stack

2-hour lab session · Visual Studio 2022 · Win32 Console Application · x86 platform

A function call is a contract. The caller puts the arguments somewhere agreed, the callee finds them there, the result comes back in `EAX`, and somebody has to tidy up afterwards. This lab writes both halves of that contract by hand.

What you will be able to do

- Refer to stack parameters and local variables by name, and know which `EBP` offsets they correspond to.
- Call a C++ function from assembly, pushing arguments right to left and cleaning up four bytes each.
- Preserve `EBX` correctly, including on a path that a conditional jump creates.
- Return a result in `EAX`, and return more than one result through pointer arguments.

How the session is organised

0:00 – 0:25 Part A. Type the lecture program in, build it, run it, and read the notes.

0:25 – 1:10 Task 1. Extend the program.

1:10 – 1:50 Task 2. Extend it again.

1:50 – 2:00 Work through the checklist at the end and ask about anything that did not build.

> **TYPE IT, DO NOT PASTE IT**
>
> Typing catches the syntax details that reading skips over — the missing comma, the wrong register width, the label with no instruction after it. The solutions at the end of this handout are for checking your work, not for copying.

Before you start

1. Open Visual Studio 2022 and choose **File > New > Project > Console App (C++)**. Give it the name shown at the top of this handout.
2. Set the **Solution Platforms** dropdown on the toolbar to **x86**. It defaults to x64, where the `_asm` keyword is rejected outright with error C4235.
3. Replace the generated `main` with the Part A listing, and make sure `#include <cstdio>` is at the top — `printf` and `scanf_s` come from there.
4. Press **Ctrl + F5** to build and run without the debugger, so the console window stays open when the program finishes.

> **IF THE PLATFORM IS WRONG**
>
> With the platform left at x64 the compiler reports that the `_asm` keyword is not supported on this architecture (C4235). Inline assembly exists on 32-bit x86 and nowhere else.

Part A — the lecture program

`absDiff(a, b)` returns the absolute difference between two integers. It exercises stack parameters, a jump on the sign flag, saving `EBX`, and returning a result in `EAX`. The lab version calls it twice: once the ordinary way, and once by hand from an `_asm` block, so you can see exactly what a call costs.

The stack frame, once the compiler has built it

```
[ebp+12]   b               second argument
[ebp+8]    a               first argument
[ebp+4]    return address
[ebp]      saved EBP       <- EBP points here
[ebp-4]    result          local variable
```

You never have to write those offsets. Name a, b and result, and the compiler substitutes the right offset for you.

The program

```
#include <cstdio>

int absDiff(int a, int b)
{
    int result;

    _asm {
        push    ebx                 // EBX is callee-saved: preserve it first

        mov     eax, a              // eax = a          (from [ebp+8])
        mov     ebx, b              // ebx = b          (from [ebp+12])
        sub     eax, ebx            // eax = a - b, and the flags now describe it

        jns     done                // SF = 0 -> already positive or zero
        neg     eax                 // otherwise flip the sign

    done:
        mov     result, eax         // store into the local          ([ebp-4])

        pop     ebx                 // restored on EVERY path out of the block
    }

    return result;                  // clear, readable, and portable
}

void partA()
{
    int fromCpp;
    int fromAsm;

    printf("--- Part A: absDiff using stack parameters ---\n\n");

    // Called the ordinary way - C++ pushes the arguments for us.
    fromCpp = absDiff(10, 3);

    // Called by hand, so you can see exactly what a call costs.
    _asm {
        push    3                   // LAST argument is pushed FIRST
        push    10                  // so 10 ends up nearest the top of the
                                    //   stack, where the callee looks for
                                    //   its first parameter at [ebp+8]
        call    absDiff
        add     esp, 8              // __cdecl: the CALLER removes the
                                    //   arguments - 2 x 4 bytes
        mov     fromAsm, eax        // the result comes back in EAX
    }

    printf("  |10 - 3| called from C++       = %d\n", fromCpp);
    printf("  |10 - 3| called from assembly  = %d\n", fromAsm);
    printf("  |3 - 10| (arguments swapped)   = %d\n", absDiff(3, 10));
    printf("\nBoth routes run exactly the same instructions.\n");
}
```

Expected output

```
|10 - 3| called from C++       = 7
|10 - 3| called from assembly  = 7
|3 - 10| (arguments swapped)   = 7
```

What to notice

- The **last** argument is pushed **first**, so the first argument ends up nearest the top of the stack — which is where the callee looks for it, at `[ebp+8]`.
- `__cdecl` makes cleanup the **caller's** job. Two arguments means `add esp, 8`. Omit it and the stack drifts a little further out of balance on every call.
- `SUB` leaves flags that describe the result, so `JNS` can ask "was that negative?" without a separate comparison.
- `POP EBX` sits after the label, so it runs whether or not the `JNS` was taken. Every push needs exactly one pop on **every** path through the block.
- The result is stored in a local and returned with an ordinary C++ `return`. MSVC would also return whatever is left in `EAX`, but the explicit form is obvious to any reader.

Task 1 — a function with three parameters

What to write

- Write `int maxOf3(int a, int b, int c)` with an `_asm` body, returning the largest of the three.
- Call it from an `_asm` block in another function, pushing all three arguments yourself.
- Print the result.

Expected output

```
maxOf3(10, 3, 25) = 25
maxOf3(-5, -20, -1) = -1
```

Hints

- Inside the function, use the running-maximum pattern from Lab 1. `EAX`, `ECX` and `EDX` are all free to use, so no pushes are needed.
- At the call site, push right to left: `push c`, then `push b`, then `push a`.
- Three arguments means `add esp, 12`, not 8. This is the single most common slip in this lab.
- The third parameter arrives at `[ebp+16]` — but you still write `c`, and the compiler works the offset out.

Task 2 — return two results through pointers

What to write

- Write `void minMax(int a, int b, int* lo, int* hi)` with an `_asm` body.
- Write the smaller of the two values into `*lo` and the larger into `*hi`.
- Call it with `&lo` and `&hi` from C++ and print both results.

Expected output

```
minMax(10, 3)   -> lo = 3, hi = 10
minMax(-7, 42)  -> lo = -7, hi = 42
minMax(5, 5)    -> lo = 5, hi = 5
```

Hints

- `EAX` can only bring one value home. Anything beyond the first result has to travel through memory, which is what a pointer argument is for.
- `lo` and `hi` are **addresses**. Load one with `mov ebx, lo` — `MOV`, not `LEA`, because the pointer's value is already the address you want.
- Then write through it with `mov [ebx], eax`. The brackets mean "go to that address and store there". Leave them off and you overwrite the pointer instead of the thing it points at.
- `XCHG EAX, EDX` swaps two registers in one instruction, which makes "put the smaller one in `EAX`" a two-line job.

Before you leave

- The Solution Platform reads **x86**, not x64.
- Every hand-written `CALL` is followed by an `ADD ESP` of four bytes per argument — 8 for two, 12 for three.

- `EBX` is popped on every path out of every block, including the ones a conditional jump creates.
- The pointer task writes through the pointer with brackets, not to the pointer itself.
- You have tested with negative values and with two equal values.

If something will not build

- `_asm not supported on this architecture` — the platform is still x64. Change it to x86.
- `improper operand type` — the operand size is ambiguous. Add `DWORD PTR`, or check you have not mixed `eax` with `al`.
- `undeclared identifier` inside a block — a misspelt variable name, or one not in scope. Names are **case sensitive** inside `_asm`, even though instruction mnemonics are not.
- The program builds but crashes *after* the block — the stack is unbalanced, or `EBX`, `ESI`, `EDI` or `EBP` was not restored.
- Right answer, wrong sign — an unsigned jump was used on signed data, or `SHR` where `SAR` was needed.

Solutions

Complete, commented code for both tasks. The Part A listing earlier in this handout is already the full solution for that section. Everything here matches the `CSI101_Lab04_Solution` project in the accompanying Visual Studio package line for line.

Task 1 — a function with three parameters

```
int maxOf3(int a, int b, int c)
{
    int result;

    _asm {
        mov     eax, a              // running maximum starts as a
        cmp     eax, b
        jge     checkC              // a >= b, so keep a
        mov     eax, b              // b wins so far

    checkC:
        cmp     eax, c
        jge     haveMax             // current best still wins
        mov     eax, c              // c wins

    haveMax:
        mov     result, eax
    }

    return result;
}

void task1()
{
    int answer;
    int a = 10, b = 3, c = 25;

    printf("\n--- Task 1: a three-parameter function ---\n\n");

    _asm {
        push    c                   // third argument pushed first
        push    b
        push    a                   // first argument pushed last
        call    maxOf3
        add     esp, 12             // THREE arguments now: 3 x 4 = 12 bytes
        mov     answer, eax
    }

    printf("  maxOf3(%d, %d, %d) = %d\n", a, b, c, answer);
    printf("  maxOf3(-5, -20, -1) = %d\n", maxOf3(-5, -20, -1));
    printf("\nThree arguments means ADD ESP, 12. Get that number wrong and the\n");
    printf("stack drifts a little further out of balance on every call.\n");
}
```

Notes on this solution

- The function contains an `_asm` block, so the compiler will not inline it. That is a limitation in general, but here it is convenient: the `CALL` you wrote by hand really is a call.
- Writing the C++ name directly in `call maxOf3` works because the compiler resolves the name to the right address at link time. You do not have to know anything about name mangling.

Task 2 — return two results through pointers

```
void minMax(int a, int b, int* lo, int* hi)
{
    _asm {
        push    ebx

        mov     eax, a
        mov     edx, b
```

```
        cmp     eax, edx
        jle     inOrder             // a <= b, so eax is already the smaller
        xchg    eax, edx            // one instruction swaps the two registers

    inOrder:                        // eax = smaller, edx = larger

        mov     ebx, lo             // ebx = the VALUE of the pointer,
                                    //   which is the address to write to
        mov     [ebx], eax          // *lo = smaller
                                    // the brackets mean "go to that address"

        mov     ebx, hi
        mov     [ebx], edx          // *hi = larger

        pop     ebx
    }
}

void task2()
{
    int lo = 0, hi = 0;

    printf("\n--- Task 2: returning results through pointers ---\n\n");

    minMax(10, 3, &lo, &hi);
    printf("  minMax(10, 3)   -> lo = %d, hi = %d\n", lo, hi);

    minMax(-7, 42, &lo, &hi);
    printf("  minMax(-7, 42)  -> lo = %d, hi = %d\n", lo, hi);

    minMax(5, 5, &lo, &hi);
    printf("  minMax(5, 5)    -> lo = %d, hi = %d\n", lo, hi);

    printf("\nMOV EBX, lo loads the pointer. MOV [EBX], EAX writes through it.\n");
    printf("Forgetting the brackets would overwrite the pointer, not the target.\n");
}
```

Notes on this solution

- Test `minMax(5, 5)`. Equal inputs go down the `JLE` path, so `lo` and `hi` both end up as 5 — correct, but worth confirming rather than assuming.
- If you see one plausible answer and one wild number, you have almost certainly written `mov ebx, [lo]` or forgotten the brackets on the store. The Memory window, with the address of `lo` typed into the address box, shows it immediately.